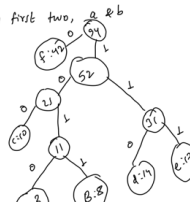
Part I:

symbol	Frequency
a	3
b	8
c	10
d	14
e	17
f	42

Increasing order frequency

Taking first two, a & b



So,

Final Bit code based on Huffman encoding!

F = 0
E = 111
D = 110
C = 100
B = 1011
A = 1010

Total cost = 1X42 + 3X17 + 3X14 + 3X10 + 4X8 + 4X3

= 209

Part II:

We already know that using priority queue makes the whole algorithm take $O(n\log n)$ time for 'n' symbols. Here, priority queue gives us $o(\log n)$ for extracting min from the queue. Insertion of the summed value also takes $O(n)$ time, making the complete algorithm $O(n\log n)$ time.

Now looking at alternative data structures:

1) Sorting before each step & taking first two.

→ This one would be same even in the priority queue, we look for the smallest frequencies & sorting & pulling initial two things to add to the tree is technically the same thing.

But the problem here is the waste of time in sorting. In avg, sorting takes $O(n\log n)$ & then we have to do the pulling out a (first two) in constant time & build the actual tree which makes the whole algorithm to be $O(n^2\log n)$ since it has to run $(n-1)$ times for $o(n\log n)$.

(2) Essentially, it is also the same since we'll have to select the first 2 min values to add to the tree but getting those values is not as cheap as using priority queue.

Even though the worst case to select a single item could go to $o(n^2)$, the expected is $o(n)$. And once the operation has to run $(n-1)$ time, our complexity becomes $O(n^2)$ which is slower than using our priority queue.

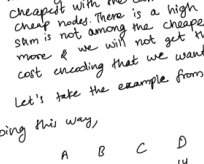
(3) Sort the symbols by frequency at the start. Merge the first two, the repeatedly merge the result with next symbol in the sorted list.

→ It's wrong. At each step we have to compare the two cheapest frequencies. But if we do this way, we'll always be comparing our initial cheapest with the current sum of previous two cheap nodes. There is a high chance that the sum is not among the cheapest (top 2) any more & we will not get the correct tree we want.

cost encoding that we want.

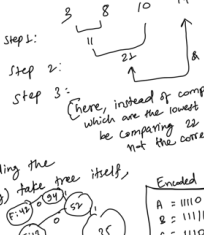
Let's take the example from Part I:

Doing this way,



There, instead of comparing 14 & 17, which are the lowest frequency, we'd be comparing 22 & 14, which is not the correct way.

Building the (wrong) take tree itself,



Encoded	cost
A = 1110	5x3 = 15
B = 1111	5x8 = 40
C = 1110	4x10 = 40
D = 110	3x14 = 2
E = 10	2x17 = 34
F = 0	1x42 =
	5 223

⇒ we can see that 223 is more than 209 which we got from Part I, so this method is incorrect.

Part 3

1. I like the first approach as well since it's very easy to retrive the value later on. But,

Storing a dictionary that maps each symbol directly to its bitstring can take $O(n^2)$ space in the worst case since each symbol's bit string can be up to $o(n)$ bits long.

Instead, using dictionary mapping each symbol to leaf node can be smart way. Each tree contains n leaf nodes & $(n-1)$ internal nodes giving total of $2n-1$ nodes. This makes space of $o(n)$ and the dictionary stores only one node reference per symbol requiring $o(n)$ space.

Therefore, this approach reduces the worst-case space complexity from $o(n^2)$ to $o(n)$ by storing tree vs duplicating complete bit strings.

But retriving will take some time we need to follow the parents & record 0 for left & 1 for a right child and reverse the bits. (little work here)